

Absolutely. We'll continue the handbook.

Company Brain Engineering Handbook

Chapter 1 – Enterprise Architecture (Part 2B)

Embedding Worker • Redis ACK/NACK • Failure Recovery •
pgvector • Retrieval • Production Scaling

11. Embedding Worker

At this point:

```
Module 1
✓ Completed

↓

Module 2

Content Normalizer
✓

Document Context Builder
✓

Language Detection
✓

Semantic Chunking
✓

Chunk Validation
✓
```

Now PostgreSQL contains something like this.

Chunks Table

chunk_id	document_id	text	importance
1	101	Revenue increased by 42%	0.95
2	101	North America sales...	0.88

chunk_id	document_id	text	importance
3	101	Operating expenses...	0.91

Notice something.

These are still **plain text**.

AI cannot search them semantically yet.

We need embeddings.

What is an Embedding Worker?

Just like the Extraction Worker...

the Embedding Worker is another Celery Worker.

```
celery \
-A app.celery_app \
worker \
--queues=embedding_queue
```

It only listens to

```
embedding_queue
```

Folder Structure

```
app/

|

├─ workers/

|   tasks.py

|

├─ embeddings/

|   bge_embedder.py

|   embedding_cache.py
```

```
| embedding_validator.py  
  
| embedding_orchestrator.py
```

Worker Code

Python

Run

```
@celery.task(  
    bind=True,  
    autoretry_for=(Exception,),  
    retry_backoff=True  
)  
def embed_document(  
    self,  
    document_id: int  
):  
  
    from app.embeddings.embedding_orchestrator import (  
        EmbeddingOrchestrator  
    )  
  
    orchestrator = EmbeddingOrchestrator()  
  
    orchestrator.process(document_id)
```

Notice

Worker

↓

Orchestrator

↓

Embedder

↓

Validator

↓

Database

Worker itself contains almost no business logic.

Embedding Orchestrator

Instead of directly calling the embedding model...

we use

```
EmbeddingOrchestrator
```

Why?

Suppose tomorrow you change

```
BGE
```

```
↓
```

```
Voyage AI
```

Only one class changes.

Everything else remains identical.

Simplified code

Python

Run

```
class EmbeddingOrchestrator:

    def process(self, document_id):

        chunks = load_chunks(document_id)

        embeddings = []

        for chunk in chunks:

            vector = embed(chunk.text)

            embeddings.append(vector)

        save_vectors(embeddings)
```

Real production code would also:

- batch requests
 - check cache
 - validate vectors
 - retry failures
 - log metrics
-

BGE Embedder

Library

Python

Run

```
from sentence_transformers import SentenceTransformer
```

Load model

Python

Run

```
model = SentenceTransformer(  
    "BAAI/bge-large-en-v1.5"  
)
```

Embedding

Python

Run

```
vector = model.encode(text)
```

Output

```
[  
  0.14,  
 -0.88,  
  0.52,  
  ...  
1024 values  
]
```

This is called

Dense Vector

Why 1024 dimensions?

Each number captures part of the sentence's meaning.

Example

Revenue increased by 42%.

becomes

```
[0.42,  
-0.15,  
0.98,  
...  
1024]
```

Another sentence

Sales grew by forty two percent.

becomes

```
[0.40,  
-0.12,  
1.02,  
...  
1024]
```

Although the wording is different, the vectors are close together.

That is semantic search.

Embedding Validator

Before saving

validate

Python

Run

```
def validate(vector):  
  
    assert len(vector)==1024  
  
    assert not np.isnan(vector).any()
```

Why?

Sometimes

GPU

↓

Memory Error

↓

Corrupted Vector

Better to detect now.

Tenant Cache

Suppose

same document

uploaded twice.

Should we compute embeddings again?

No.

Cache key

```
tenant_id  
  
+  
  
SHA256(chunk_text)
```

If found

↓

Reuse

Otherwise

↓

Generate

Store Embeddings

Now save

Chunk

↓

Vector

Table

```
embeddings
-----

chunk_id

embedding VECTOR(1024)

model_name

created_at

-----
```

Notice

We store

chunk_id

not

document text.

The text already exists

inside

chunks table.

pgvector

Why

pgvector?

Because

PostgreSQL

cannot normally understand vectors.

Extension

```
CREATE EXTENSION pgvector;
```

adds

```
VECTOR
```

datatype.

Example

```
embedding
```

```
VECTOR(1024)
```

HNSW Index

Without index

Searching

```
20 million vectors
```

requires

checking

every vector.

Impossible.

Instead

HNSW

Index

reduces

search

to milliseconds.

Create index

SQL

```
CREATE INDEX idx_embedding  
  
ON embeddings  
  
USING hnsw  
  
(  
  
embedding  
  
vector_cosine_ops  
  
);
```

Now

search

is fast.

Retrieval

Employee asks

What was the revenue growth?

Flow

Question

↓

Embedding Model

↓

Question Vector

↓

pgvector Search

↓

Nearest Chunks

↓

AI

SQL

SQL

```
SELECT
```

```
chunk_id,
```

```
embedding <=> query_vector
```

```
AS distance
```

```
FROM embeddings
```

```
ORDER BY distance
```

```
LIMIT 10;
```

Notice

<=>

Cosine Distance.

12. ACK & NACK

Now let's understand

Celery internally.

Suppose Redis

contains

Task

↓

embed_document

Worker

asks

Any work?

Redis replies

Take Task.

Worker starts.

ACK

ACK

means

Acknowledged

Worker says

I finished successfully.

Redis

removes task forever.

NACK

Suppose

GPU crashes.

Worker says

```
I couldn't finish.
```

This is effectively a negative acknowledgement or failure state, depending on broker and configuration.

Task returns

to queue

or is retried according to Celery's retry policy.

Worker Crash

Suppose

```
Embedding
```

```
↓
```

```
50%
```

```
↓
```

```
Server crashes
```

Without retries

```
↓
```

Task lost.

With

```
acks_late=True
```

the task is acknowledged **only after successful completion**.

If the worker dies before then,

another worker

can process it.

Example:

Python

Run

```
@celery.task(
    bind=True,
    acks_late=True
)
```

This improves reliability, though you must also make the task idempotent so it is safe to run more than once.

Retries

Suppose

OpenAI

↓

Timeout

Celery retries.

Attempt 1

↓

Fail

↓

Wait

↓

Attempt 2

↓

Success

Idempotency

Very important.

Suppose

Worker crashes

after storing vectors.

Retry happens.

Should we insert

duplicate vectors?

No.

Always check

```
chunk_id
```

```
+
```

```
model_name
```

already exists.

If yes

↓

Skip.

Idempotency ensures running the same task twice produces the same final state.

Complete Pipeline

```
Upload PDF
```

```
↓
```

```
FastAPI
```

```
↓
```

```
S3
```


↓

Embedding Worker

↓

BGE Model

↓

Embedding Validator

↓

pgvector

↓

Retrieval Ready

Scaling

Suppose

1 document

One worker

is enough.

Suppose

100,000 documents

Need

10 Extraction Workers

20 Chunk Workers

50 GPU Embedding Workers

Each queue scales independently.

This is why

microservices

and

Celery

are powerful.

Monitoring

Every worker should emit metrics.

Examples

Extraction Time

Chunk Count

Embedding Time

GPU Usage

Queue Length

Worker Failures

Retry Count

Average Retrieval Time

Libraries

prometheus_client

OpenTelemetry

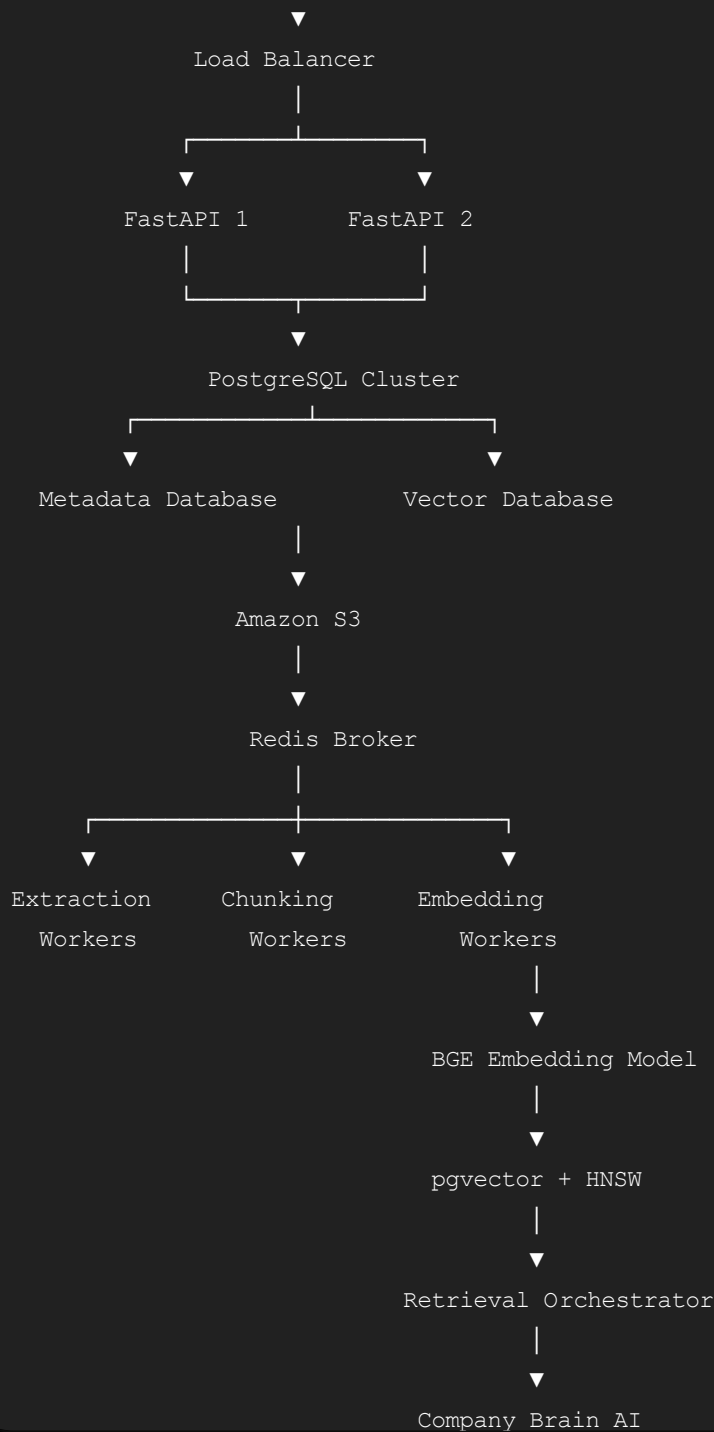
Grafana

CloudWatch

Production Architecture

Users

|



Chapter 1 Summary

By the end of this chapter, you now understand:

- Why FastAPI only coordinates requests.
- Why S3 stores the original documents.
- How AWS automatically encrypts and decrypts files with SSE-KMS.
- Why PostgreSQL stores structured enterprise data.
- Why Redis is used as a message broker instead of a database.
- How Celery serializes tasks and workers execute your Python functions.
- How Module 1 hands work to Module 2 using task chaining.

-  How the Embedding Worker generates and validates vectors.
 -  How `pgvector` and HNSW make semantic search efficient.
 -  Why acknowledgements, retries, and idempotency are critical for reliable distributed systems.
 -  How the complete pipeline scales from a single document to millions of documents.
-

My recommendation before Chapter 2

This chapter explains **how the system works**.

Before moving on, I recommend one additional chapter that most tutorials skip:

"Building Company Brain Locally from Scratch."

In that chapter, we'll create the entire project from an empty folder, install every dependency, configure Docker, PostgreSQL, Redis, Celery, S3-compatible storage (such as MinIO for local development), and run the first end-to-end pipeline. That way, when we begin implementing Module 2 in depth, you'll already have a working development environment that mirrors the production architecture. This will make every later chapter much easier to follow and implement.